02. Station 1: Gatekeeper (`capture.py`)

1. Overview & Purpose

`capture.py` is the **first defense line (Gatekeeper)** of the sensing layer.

On a busy enterprise network, thousands of packets cross the NIC every second (UDP video streams, ICMP pings, ARP requests, broadcast discovery). Attempting to extract complex TLS handshakes from non-TCP traffic would crash the parser and waste CPU cycles.

`capture.py` filters raw frames with minimal overhead:

- It supports **Dual-Stack** networking (IPv4 and IPv6).
- It verifies that the packet contains an IP layer and a TCP transport layer.
- If either is missing, it drops the packet immediately by returning `None`.
- If valid, it returns the raw packet for deeper inspection in `ja4.py`.

2. Complete Source Code

```
python
from scapy.all import sniff, IP, TCP, IPv6

def capture(packet):
    # Check for IP (v4 or v6) and TCP
    if not ((packet.haslayer(IP) or packet.haslayer(IPv6)) and packet.haslayer(TCP)):
        return None
    return packet

if __name__ == "__main__":
    sniff(iface="any", filter="tcp port 443 or tcp port 80", prn=capture, store=0)
```

3. Function-by-Function Breakdown

Function: `capture(packet)`

- **Input:** `packet` (A raw packet object provided by Scapy's packet sniffer).
- **Output:** Returns the `packet` if it contains IP + TCP; returns `None` if it does not.
- **How it works:**
    1. `packet.haslayer(IP)` checks if IPv4 headers exist (`0x0800`).
    2. `packet.haslayer(IPv6)` checks if IPv6 headers exist (`0x86DD`).
    3. `packet.haslayer(TCP)` checks if the transport protocol is TCP (protocol 6).

4. The logical condition `((IP or IPv6) and TCP)` guarantees that only routable TCP packets reach our feature extraction station.

4. Line-by-Line Explanation

Line	Code	Explanation in Simple Terms
1	`from scapy.all import sniff, IP, TCP, IPv6`	Imports Scapy's packet sniffer and protocol layer classes for IPv4, TCP, and IPv6 header recognition.
3	`def capture(packet):`	Defines the gatekeeper callback function which receives each incoming frame off the wire.
5	`if not ((packet.haslayer(IP) or packet.haslayer(IPv6)) and packet.haslayer(TCP)):`	Evaluates if the packet has an IP layer (either v4 or v6) AND a TCP layer. If false, executes line 6.
6	`return None`	Drops the packet immediately. Zero further CPU cycles are spent on this packet.
7	`return packet`	Passes the validated packet forward to Station 2 (`ja4.py`).
9	`if __name__ == "__main__":`	Guard block ensuring standalone test execution only runs when executing `python3 capture.py` directly.
10	`sniff(iface="any", filter="tcp port 443 or tcp port 80", prn=capture, store=0)`	Starts a standalone capture test listening on all network interfaces for HTTP/HTTPS TCP traffic, passing each packet to `capture()` .

5. Variables & System Mapping

Variable Name	Type	Scope	Role in this File	Downstream Mapping (Backend / Database)
`packet`	`scapy.layers.l2.Ether` or `Packet`	Local (Argument)	Contains the raw binary frames captured by the network card.	Passed directly into `ja4.py:process_packet()` to extract metadata. Not sent across HTTP raw.
`iface`	`str`	Argument to `sniff()`	Network interface to listen on (e.g., `eth0` , `any`).	None. Internal to capture layer.
`filter`	`str`	Argument to `sniff()`	Berkeley Packet Filter (BPF) syntax telling the OS kernel which packets to hand to Python.	None. Internal to OS kernel socket.
`store`	`int`	Argument to `sniff()`	`0` tells Scapy not to buffer packets in memory.	None. Critical guard against memory exhaustion.